

Some Math behind Neural Tangent Kernel

This PDF conversion (v1) was generated on October 10, 2025*

Online version: <https://lilianweng.github.io/posts/2022-09-08-ntk/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2022-09-08-ntk/>.

Contents

1	Some Math behind Neural Tangent Kernel	3
2	Basics#	4
2.1	Vector-to-vector Derivative#	4
2.2	Differential Equations#	4
2.3	Central Limit Theorem#	5
2.4	Taylor Expansion#	5
2.5	Kernel & Kernel Methods#	5
2.6	Gaussian Processes#	6
3	Notation#	6
4	Neural Tangent Kernel#	6
5	Infinite Width Networks#	7
5.1	Connection with Gaussian Processes#	7
5.2	Deterministic Neural Tangent Kernel#	9
5.3	Linearized Models#	11
5.4	Lazy Training#	12
6	Citation#	12
7	References#	13
8	Citation	13
9	References	15

[Lil'Log](#)

- [|](#)
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 Some Math behind Neural Tangent Kernel

Date: September 8, 2022 | Estimated Reading Time: 17 min | Author: Lilian Weng
Table of Contents

- [Basics](#)
 - [Vector-to-vector Derivative](#)
 - [Differential Equations](#)
 - [Central Limit Theorem](#)
 - [Taylor Expansion](#)
 - [Kernel & Kernel Methods](#)
 - [Gaussian Processes](#)
- [Notation](#)
- [Neural Tangent Kernel](#)
- [Infinite Width Networks](#)
 - [Connection with Gaussian Processes](#)
 - [Deterministic Neural Tangent Kernel](#)
 - [Linearized Models](#)
 - [Lazy Training](#)
- [Citation](#)
- [References](#)

Neural networks are [well known](#) to be over-parameterized and can often easily fit data with near-zero training loss with decent generalization performance on test dataset. Although all these parameters are initialized at random, the optimization process can consistently lead to similarly good outcomes. And this is true even when the number of model parameters exceeds the number of training data points.

Neural tangent kernel (NTK) ([Jacot et al. 2018](#)) is a kernel to explain the evolution of neural networks during training via gradient descent. It leads to great insights into why neural networks with enough width can consistently converge to a global minimum when trained to minimize an empirical loss. In the post, we will do a deep dive into the motivation and definition of NTK, as well as the proof of a deterministic convergence at different initializations of neural networks with infinite width by characterizing NTK in such a setting.

Different from my previous posts, this one mainly focuses on a small number of core papers, less on the breadth of the literature review in the field. There are many interesting works after NTK, with modification or expansion of the theory for understanding the learning dynamics of NNs, but they won't be covered here. The goal is to show all the math behind NTK in a clear and easy-to-follow format, so the post is quite math-intensive. If you notice any mistakes, please let me know and I will be happy to correct them quickly. Thanks in advance!

2 Basics#

This section contains reviews of several very basic concepts which are core to understanding of neural tangent kernel. Feel free to skip.

2.1 Vector-to-vector Derivative#

Given an input vector $\mathbf{x} \in \mathbb{R}^n$ (as a column vector) and a function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, the derivative of f with respect to $\mathbf{x}$ is a $m \times n$ matrix, also known as *Jacobian matrix*:

$$J = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

Throughout the post, I use integer subscript(s) to refer to a single entry out of a vector or matrix value; i.e. x_i indicates the i -th value in the vector $\mathbf{x}$ and $f_i(\cdot)$ is the i -th entry in the output of the function.

The gradient of a vector with respect to a vector is defined as $\nabla_{\mathbf{x}} f = J^{\top} \in \mathbb{R}^{n \times m}$ and this formation is also valid when $m=1$ (i.e., scalar output).

2.2 Differential Equations#

Differential equations describe the relationship between one or multiple functions and their derivatives. There are two main types of differential equations.

- (1) *ODE (Ordinary differential equation)* contains only an unknown function of one random variable. ODEs are the main form of differential equations used in this post. A general form of ODE looks like $(x, y, \frac{dy}{dx}, \dots, \frac{d^ny}{dx^n}) = 0$.
- (2) *PDE (Partial differential equation)* contains unknown multivariable functions and their partial derivatives.

Let's review the simplest case of differential equations and its solution. *Separation of variables* (Fourier method) can be used when all the terms containing one variable can be moved to one side, while the other terms are all moved to the other side. For example,

$$\begin{aligned} &\text{\texttt{\textbackslash begin{aligned} \text{Given } a \text{ is a constant scalar:} \quad \frac{dy}{dx} \\ &\&= ay \quad \text{\texttt{\textbackslash text{Move same variables to the same side:}} \quad \frac{dy}{y} \&= a dx \quad \text{\texttt{\textbackslash text{Put integral on both sides:}} \quad \int \frac{dy}{y} \&= \int a dx \quad \text{\texttt{\textbackslash text{Finally}} \quad y \&= e^{ax + C} = C e^{ax} \text{\texttt{\textbackslash end{aligned}}} \end{aligned}}$$

2.3 Central Limit Theorem#

Given a collection of i.i.d. random variables, $x_1, \dots, x_N$ with mean μ and variance σ^2 , the *Central Limit Theorem (CTL)* states that the expectation would be Gaussian distributed when N becomes really large.

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \sim \mathcal{N}(\mu, \frac{\sigma^2}{N}) \quad \text{as } N \rightarrow \infty$$

CTL can also apply to multidimensional vectors, and then instead of a single scale σ^2 we need to compute the covariance matrix of random variable Σ .

2.4 Taylor Expansion#

The *Taylor expansion* is to express a function as an infinite sum of components, each represented in terms of this function's derivatives. The Taylor expansion of a function $f(x)$ at $x=a$ can be written as:
$$f(x) = f(a) + \sum_{k=1}^{\infty} \frac{1}{k!} (x - a)^k \nabla_k f(x) \big|_{x=a}$$
 where ∇^k denotes the k -th derivative.

The first-order Taylor expansion is often used as a linear approximation of the function value:

$$f(x) \approx f(a) + (x - a) \nabla_x f(x) \big|_{x=a}$$

2.5 Kernel & Kernel Methods#

A *kernel* is essentially a similarity function between two data points, $K: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$. It describes how sensitive the prediction for one data sample is to the prediction for the other; or in other words, how similar two data points are. The kernel should be symmetric, $K(x, x') = K(x', x)$.

Depending on the problem structure, some kernels can be decomposed into two feature maps, one corresponding to one data point, and the kernel value is an inner product of these two features: $K(x, x') = \langle \varphi(x), \varphi(x') \rangle$.

Kernel methods are a type of non-parametric, instance-based machine learning algorithms. Assuming we have known all the labels of training samples $\{x^{(i)}, y^{(i)}\}$, the label for a new input x is predicted by a weighted sum $\sum_i K(x^{(i)}, x) y^{(i)}$.

2.6 Gaussian Processes#

Gaussian process (GP) is a non-parametric method by modeling a multivariate Gaussian probability distribution over a collection of random variables. GP assumes a prior over functions and then updates the posterior over functions based on what data points are observed.

Given a collection of data points $\{x^{(1)}, \dots, x^{(N)}\}$, GP assumes that they follow a jointly multivariate Gaussian distribution, defined by a mean $\mu(x)$ and a covariance matrix $\Sigma(x)$. Each entry at location (i, j) in the covariance matrix $\Sigma(x)$ is defined by a kernel $\Sigma_{i,j} = K(x^{(i)}, x^{(j)})$, also known as a *covariance function*. The core idea is – if two data points are deemed similar by the kernel, the function outputs should be close, too. Making predictions with GP for unknown data points is equivalent to drawing samples from this distribution, via a conditional distribution of unknown data points given observed ones.

Check [this post](#) for a high-quality and highly visualization tutorial on what Gaussian Processes are.

3 Notation#

Let us consider a fully-connected neural networks with parameter θ , $f(\cdot; \theta)$: $\mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}$. Layers are indexed from 0 (input) to L (output), each containing $n_0, \dots, n_L$ neurons, including the input of size n_0 and the output of size n_L . There are $P = \sum_{l=0}^{L-1} (n_l + 1) n_{l+1}$ parameters in total and thus we have $\theta \in \mathbb{R}^P$.

The training dataset contains N data points, $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$. All the inputs are denoted as $\mathcal{X} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ and all the labels are denoted as $\mathcal{Y} = \{y^{(i)}\}_{i=1}^N$.

Now let's look into the forward pass computation in every layer in detail. For $l=0, \dots, L-1$, each layer l defines an affine transformation $A^{(l)}$ with a weight matrix $\mathbf{w}^{(l)} \in \mathbb{R}^{n_{l-1} \times n_{l+1}}$ and a bias term $\mathbf{b}^{(l)} \in \mathbb{R}^{n_{l+1}}$, as well as a pointwise nonlinearity function $\sigma(\cdot)$ which is [Lipschitz continuous](#).

$$\begin{aligned} A^{(0)} &= \mathbf{x} \parallel \tilde{A}^{(l+1)}(\mathbf{x}) &= \frac{1}{\sqrt{n_l}} \{\mathbf{w}^{(l)}\}^\top A^{(l)} + \beta \mathbf{b}^{(l)} \quad \text{in } \mathbb{R}^{n_{l+1}} \\ &\text{; pre-activations} \\ A^{(l+1)}(\mathbf{x}) &= \sigma(\tilde{A}^{(l+1)}(\mathbf{x})) \quad \text{in } \mathbb{R}^{n_{l+1}} \\ &\text{; post-activations} \end{aligned}$$

Note that the *NTK parameterization* applies a rescale weight $1/\sqrt{n_l}$ on the transformation to avoid divergence with infinite-width networks. The constant scalar $\beta \geq 0$ controls how much effort the bias terms have.

All the network parameters are initialized as an i.i.d Gaussian $\mathcal{N}(0, 1)$ in the following analysis.

4 Neural Tangent Kernel#

Neural tangent kernel (NTK) ([Jacot et al. 2018](#)) is an important concept for understanding neural network training via gradient descent. At its core, it explains how

updating the model parameters on one data sample affects the predictions for other samples.

Let's start with the intuition behind NTK, step by step.

The empirical loss function $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$ to minimize during training is defined as follows, using a per-sample cost function ℓ :

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$$

and according to the chain rule, the gradient of the loss is:

$$\nabla_{\theta} \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \underbrace{\nabla_{\theta} f(\mathbf{x}^{(i)}; \theta)}_{\text{size } P \times n_L} \underbrace{\nabla_{\mathbf{f}} \ell(f, y^{(i)})}_{\text{size } n_L \times 1}$$

When tracking how the network parameter θ evolves in time, each gradient descent update introduces a small incremental change of an infinitesimal step size. Because of the update step is small enough, it can be approximately viewed as a derivative on the time dimension:

$$\frac{d\theta}{dt} = -\nabla_{\theta} \mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \nabla_{\theta} f(\mathbf{x}^{(i)}; \theta) \nabla_{\mathbf{f}} \ell(f, y^{(i)})$$

Again, by the chain rule, the network output evolves according to the derivative:

$$\frac{d f(\mathbf{x}; \theta)}{dt} = \frac{d f(\mathbf{x}; \theta)}{d \theta} \frac{d \theta}{dt} = -\frac{1}{N} \sum_{i=1}^N \underbrace{\nabla_{\theta} f(\mathbf{x}; \theta)}_{\text{Neural tangent kernel}} \nabla_{\mathbf{f}} \ell(f, y^{(i)})$$

Here we find the **Neural Tangent Kernel (NTK)**, as defined in the blue part in the above formula, $K(\mathbf{x}, \mathbf{x}'; \theta) = \nabla_{\theta} f(\mathbf{x}; \theta) \nabla_{\theta} f(\mathbf{x}'; \theta)$:

$$K(\mathbf{x}, \mathbf{x}'; \theta) = \nabla_{\theta} f(\mathbf{x}; \theta) \nabla_{\theta} f(\mathbf{x}'; \theta)$$

where each entry in the output matrix at location (m, n) , $1 \leq m, n \leq n_L$ is:

$$K_{m,n}(\mathbf{x}, \mathbf{x}'; \theta) = \sum_{p=1}^P \frac{\partial f_m(\mathbf{x}; \theta)}{\partial \theta_p} \frac{\partial f_n(\mathbf{x}'; \theta)}{\partial \theta_p}$$

The “feature map” form of one input $\mathbf{x}$ is $\varphi(\mathbf{x}) = \nabla_{\theta} f(\mathbf{x}; \theta)$.

5 Infinite Width Networks#

To understand why the effect of one gradient descent is so similar for different initializations of network parameters, several pioneering theoretical work starts with infinite width networks. We will look into detailed proof using NTK of how it guarantees that infinite width networks can converge to a global minimum when trained to minimize an empirical loss.

5.1 Connection with Gaussian Processes#

Deep neural networks have deep connection with gaussian processes (Neal 1994). The output functions of a L -layer network, $f_i(\mathbf{x}; \theta)$ for $i=1, \dots, n_L$

, are i.i.d. centered Gaussian process of covariance $\Sigma^{\{L\}}$, defined recursively as:

$$\begin{aligned} \Sigma^{\{1\}}(\mathbf{x}, \mathbf{x}') &= \frac{1}{n_0} \mathbf{x}^{\top} \mathbf{x}' + \beta^2 \lambda^{\{1+1\}}(\mathbf{x}, \mathbf{x}') \\ \Sigma^{\{l\}}(\mathbf{x}, \mathbf{x}') &= \begin{bmatrix} \Sigma^{\{l\}}(\mathbf{x}, \mathbf{x}) & \Sigma^{\{l\}}(\mathbf{x}, \mathbf{x}') \\ \Sigma^{\{l\}}(\mathbf{x}', \mathbf{x}) & \Sigma^{\{l\}}(\mathbf{x}', \mathbf{x}') \end{bmatrix} \\ \Sigma^{\{l+1\}}(\mathbf{x}, \mathbf{x}') &= \mathbb{E}_{\mathbf{f} \sim \mathcal{N}(0, \lambda^{\{l\}})} [\sigma(\mathbf{f}(\mathbf{x})) \sigma(\mathbf{f}(\mathbf{x}'))] + \beta^2 \end{aligned}$$

Lee & Bahri et al. (2018) showed a proof by mathematical induction:

(1) Let's start with $L=1$, when there is no nonlinearity function and the input is only processed by a simple affine transformation:

$$\begin{aligned} \mathbf{f}(\mathbf{x}; \theta) &= \tilde{\mathbf{A}}^{\{1\}}(\mathbf{x}) = \frac{1}{\sqrt{n_0}} \mathbf{w}^{\{0\}}_{\cdot i} \mathbf{x}_i + \beta b^{\{0\}}_{\cdot m} \\ \text{where } \tilde{\mathbf{A}}^{\{1\}}_m(\mathbf{x}) &= \frac{1}{\sqrt{n_0}} \mathbf{w}^{\{0\}}_{\cdot i} \mathbf{x}_i + \beta b^{\{0\}}_{\cdot m} \quad \text{for } 1 \leq m \leq n-1 \end{aligned}$$

Since the weights and biases are initialized i.i.d., all the output dimensions of this network $\{\tilde{\mathbf{A}}^{\{1\}}_1(\mathbf{x}), \dots, \tilde{\mathbf{A}}^{\{1\}}_{n-1}(\mathbf{x})\}$ are also i.i.d. Given different inputs, the m -th network outputs $\tilde{\mathbf{A}}^{\{1\}}_m(\cdot)$ have a joint multivariate Gaussian distribution, equivalent to a Gaussian process with covariance function (We know that mean $\mu_w = \mu_b = 0$ and variance $\sigma^2_w = \sigma^2_b = 1$)

$$\begin{aligned} \Sigma^{\{1\}}(\mathbf{x}, \mathbf{x}') &= \mathbb{E}[\tilde{\mathbf{A}}^{\{1\}}_m(\mathbf{x}) \tilde{\mathbf{A}}^{\{1\}}_m(\mathbf{x}')] \\ &= \mathbb{E}[\frac{1}{\sqrt{n_0}} \sum_{i=1}^{n_0} \mathbf{w}^{\{0\}}_{\cdot i} \mathbf{x}_i + \beta b^{\{0\}}_{\cdot m}][\frac{1}{\sqrt{n_0}} \sum_{i=1}^{n_0} \mathbf{w}^{\{0\}}_{\cdot i} \mathbf{x}'_i + \beta b^{\{0\}}_{\cdot m}] \\ &= \frac{1}{n_0} \sum_{i=1}^{n_0} \sum_{j=1}^{n_0} \mathbf{x}_i \mathbf{x}'_j + \frac{\beta}{\sqrt{n_0}} \sum_{i=1}^{n_0} \mathbf{w}_{\cdot i} (\mathbf{x}_i + \mathbf{x}'_i) + \sigma^2_b \beta^2 \\ &= \frac{1}{n_0} \mathbf{x}^{\top} \mathbf{x}' + \beta^2 \end{aligned}$$

(2) Using induction, we first assume the proposition is true for $L=l$, a l -layer network, and thus $\tilde{\mathbf{A}}^{\{l\}}_m(\cdot)$ is a Gaussian process with covariance $\Sigma^{\{l\}}$ and $\{\tilde{\mathbf{A}}^{\{l\}}_i\}_{i=1}^{n_l}$ are i.i.d.

Then we need to prove the proposition is also true for $L=l+1$. We compute the outputs by:

$$\begin{aligned} \mathbf{f}(\mathbf{x}; \theta) &= \tilde{\mathbf{A}}^{\{l+1\}}(\mathbf{x}) = \frac{1}{\sqrt{n_l}} \sigma(\tilde{\mathbf{A}}^{\{l\}}_i(\mathbf{x})) + \beta b^{\{l\}}_{\cdot m} \\ &= \frac{1}{\sqrt{n_l}} \sum_{i=1}^{n_l} \mathbf{w}^{\{l\}}_{\cdot i} \sigma(\tilde{\mathbf{A}}^{\{l\}}_i(\mathbf{x})) + \beta b^{\{l\}}_{\cdot m} \quad \text{for } 1 \leq m \leq n_{l+1} \end{aligned}$$

We can infer that the expectation of the sum of contributions of the previous hidden layers is zero:

$$\begin{aligned} &\mathbb{E}[\mathbf{w}^{\{l\}}_{\cdot i} \sigma(\tilde{\mathbf{A}}^{\{l\}}_i(\mathbf{x}))] \\ &= \mathbb{E}[\mathbf{w}^{\{l\}}_{\cdot i}] \mathbb{E}[\sigma(\tilde{\mathbf{A}}^{\{l\}}_i(\mathbf{x}))] = 0 \\ &= \mathbb{E}[\mathbf{w}^{\{l\}}_{\cdot i}]^2 \mathbb{E}[\sigma(\tilde{\mathbf{A}}^{\{l\}}_i(\mathbf{x}))^2] \\ &= \sigma^2_w \Sigma^{\{l\}}(\mathbf{x}, \mathbf{x}) = \Sigma^{\{l\}}(\mathbf{x}, \mathbf{x}) \end{aligned}$$

Since $\{\tilde{\mathbf{A}}^{\{l\}}_i(\mathbf{x})\}_{i=1}^{n_l}$ are i.i.d., according to central limit theorem, when the hidden layer gets infinitely wide $n_l \rightarrow \infty$, $\tilde{\mathbf{A}}^{\{l+1\}}_m(\mathbf{x})$ is Gaussian distributed with variance $\beta^2 + \text{Var}(\tilde{\mathbf{A}}^{\{l\}}_i(\mathbf{x}))$.

Note that $\{\tilde{A}^{\{(l+1)\}}_1(\mathbf{x}), \dots, \tilde{A}^{\{(l+1)\}}_{n_{l+1}}(\mathbf{x})\}$ are still i.i.d.

$\tilde{A}^{\{(l+1)\}}_m(\cdot)$ is equivalent to a Gaussian process with covariance function:

$$\begin{aligned} & \mathbb{E}[\tilde{A}^{\{(l+1)\}}_m(\mathbf{x}) \tilde{A}^{\{(l+1)\}}_n(\mathbf{x}')] = \frac{1}{n_l} \sigma_{\text{big}}(\tilde{A}^{\{(l)\}}_i(\mathbf{x}))^{\text{top}} \sigma_{\text{big}}(\tilde{A}^{\{(l)\}}_j(\mathbf{x}')) \\ & + \beta^2 \quad \text{similar to how we get } \Sigma^{\{(1)\}} \end{aligned}$$

When $n_l \rightarrow \infty$, according to central limit theorem,

$$\Sigma^{\{(l+1)\}}(\mathbf{x}, \mathbf{x}') \rightarrow \mathbb{E}_{\mathbf{f} \sim \mathcal{N}(0, \Lambda^{\{(l)\}})}[\sigma(f(\mathbf{x}))^{\text{top}} \sigma(f(\mathbf{x}'))] + \beta^2$$

The form of Gaussian processes in the above process is referred to as the *Neural Network Gaussian Process (NNGP)* (Lee & Bahri et al. (2018)).

5.2 Deterministic Neural Tangent Kernel#

Finally we are now prepared enough to look into the most critical proposition from the NTK paper:

When $n_1, \dots, n_L \rightarrow \infty$ (network with infinite width), the NTK converges to be:

- (1) **deterministic at initialization, meaning that the kernel is irrelevant to the initialization values and only determined by the model architecture; and**
- (2) **stays constant during training.**

The proof depends on mathematical induction as well:

(1) First of all, we always have $K^{\{(0)\}} = 0$. When $L=1$, we can get the representation of NTK directly. It is deterministic and does not depend on the network initialization. There is no hidden layer, so there is nothing to take on infinite width.

$$\begin{aligned} & f(\mathbf{x}; \theta) = \tilde{A}^{\{(1)\}}(\mathbf{x}) = \frac{1}{\sqrt{n_0}} \{\mathbf{w}^{\{(0)\}}\}^{\text{top}} \mathbf{x} + \mathbf{b}^{\{(0)\}} \\ & K^{\{(1)\}}(\mathbf{x}, \mathbf{x}'; \theta) = \mathbb{E}[\frac{\partial f(\mathbf{x}; \theta)}{\partial \mathbf{w}^{\{(0)\}}} \frac{\partial f(\mathbf{x}'; \theta)}{\partial \mathbf{w}^{\{(0)\}}} + \mathbb{E}[\frac{\partial f(\mathbf{x}; \theta)}{\partial \mathbf{b}^{\{(0)\}}} \frac{\partial f(\mathbf{x}'; \theta)}{\partial \mathbf{b}^{\{(0)\}}}] \\ & = \frac{1}{n_0} \mathbf{x}^{\text{top}} \mathbf{x}' + \beta^2 = \Sigma^{\{(1)\}}(\mathbf{x}, \mathbf{x}') \end{aligned}$$

(2) Now when $L=1$, we assume that a 1-layer network with P parameters in total, $\tilde{\theta} = (\mathbf{w}^{\{(0)\}}, \dots, \mathbf{w}^{\{(l-1)\}}, \mathbf{b}^{\{(0)\}}, \dots, \mathbf{b}^{\{(l-1)\}}) \in \mathbb{R}^{\tilde{P}}$, has a NTK converging to a deterministic limit when $n_1, \dots, n_{l-1} \rightarrow \infty$.

$$K^{\{(1)\}}(\mathbf{x}, \mathbf{x}'; \tilde{\theta}) = \nabla_{\tilde{\theta}} \tilde{A}^{\{(1)\}}(\mathbf{x}) \nabla_{\tilde{\theta}} \tilde{A}^{\{(1)\}}(\mathbf{x}') \rightarrow K^{\{(1)\}}_{\infty}(\mathbf{x}, \mathbf{x}')$$

Note that $K_{\infty}^{\{(1)\}}$ has no dependency on θ .

Next let's check the case $L=l+1$. Compared to a 1-layer network, a $(l+1)$ -layer network has additional weight matrix $\mathbf{w}^{\{(l)\}}$ and bias $\mathbf{b}^{\{(l)\}}$ and thus the total parameters contain $\theta = (\tilde{\theta}, \mathbf{w}^{\{(l)\}}, \mathbf{b}^{\{(l)\}})$.

The output function of this $(l+1)$ -layer network is:

$$f(\mathbf{x}; \theta) = \frac{1}{\sqrt{n}} \sum_{i=1}^n \tilde{A}_i(\mathbf{x}) \mathbf{w}_i + \beta \mathbf{b}$$

And we know its derivative with respect to different sets of parameters; let denote $\tilde{A}_i(\mathbf{x}) = \tilde{A}_i(\mathbf{x})$ for brevity in the following equation:

$$\begin{aligned} \nabla_{\mathbf{w}} f(\mathbf{x}; \theta) &= \frac{1}{\sqrt{n}} \sum_{i=1}^n \tilde{A}_i(\mathbf{x}) \mathbf{w}_i \\ \nabla_{\beta} f(\mathbf{x}; \theta) &= \mathbf{b} \\ \nabla_{\theta} f(\mathbf{x}; \theta) &= \frac{1}{\sqrt{n}} \sum_{i=1}^n \tilde{A}_i(\mathbf{x}) \nabla_{\theta} \tilde{A}_i(\mathbf{x}) \end{aligned}$$

where ∇_{θ} is the derivative of θ and each entry at location (p, m) , $1 \leq p \leq P$, $1 \leq m \leq n+1$ in the matrix ∇_{θ} can be written as

$$\frac{\partial f_m(\mathbf{x}; \theta)}{\partial \theta_p} = \sum_{i=1}^n \tilde{A}_i(\mathbf{x}) \frac{\partial \tilde{A}_i(\mathbf{x})}{\partial \theta_p}$$

The NTK for this $(l+1)$ -layer network can be defined accordingly:

$$\begin{aligned} K(\mathbf{x}, \mathbf{x}') &= \nabla_{\theta} f(\mathbf{x}; \theta) \nabla_{\theta} f(\mathbf{x}'; \theta) \\ &= \frac{1}{n} \sum_{i,j=1}^n \tilde{A}_i(\mathbf{x}) \tilde{A}_j(\mathbf{x}') \nabla_{\theta} \tilde{A}_i(\mathbf{x}) \nabla_{\theta} \tilde{A}_j(\mathbf{x}') \\ &= \frac{1}{n} \sum_{i,j=1}^n \tilde{A}_i(\mathbf{x}) \tilde{A}_j(\mathbf{x}') \mathbf{K}_{ij} \end{aligned}$$

where each individual entry at location (m, n) , $1 \leq m, n \leq n+1$ of the matrix $\mathbf{K}$ can be written as:

$$\begin{aligned} K^{\{(l+1)\}}_{mn} &= \frac{1}{n_l} \Big[\sum_{i=1}^{n_l} \sum_{j=1}^{n_l} w^{\{(l)\}}_{im} w^{\{(l)\}}_{jn} \dot{\sigma}(\tilde{A}_{-i}^{\{(l)\}}) \dot{\sigma}(\tilde{A}_{-j}^{\{(l)\}}) K_{ij}^{\{(l)\}} \Big] \end{aligned}$$

When $n_l \rightarrow \infty$, the section in blue and green has the limit (See the proof in the [previous section](#)):

$$\frac{1}{n_l} \sigma(\tilde{A}^{\{(l)\}}) \sigma(\tilde{A}^{\{(l)\}}) + \beta^2 \rightarrow \Sigma^{\{(l+1)\}}$$

and the red section has the limit:

$$\sum_{i=1}^{n_l} \sum_{j=1}^{n_l} w^{\{(l)\}}_{im} w^{\{(l)\}}_{jn} \dot{\sigma}(\tilde{A}_{-i}^{\{(l)\}}) \dot{\sigma}(\tilde{A}_{-j}^{\{(l)\}}) K_{ij}^{\{(l)\}} \rightarrow \sum_{i=1}^{n_l} \sum_{j=1}^{n_l} w^{\{(l)\}}_{im} w^{\{(l)\}}_{jn} \dot{\sigma}(\tilde{A}_{-i}^{\{(l)\}}) \dot{\sigma}(\tilde{A}_{-j}^{\{(l)\}}) K_{ij}^{\{\infty, ij\}}$$

Later, [Arora et al. \(2019\)](#) provided a proof with a weaker limit, that does not require all the hidden layers to be infinitely wide, but only requires the minimum width to be sufficiently large.

5.3 Linearized Models#

From the [previous section](#), according to the derivative chain rule, we have known that the gradient update on the output of an infinite width network is as follows; For brevity, we omit the inputs in the following analysis:

$$\frac{df(\theta)}{dt} = -\eta \nabla_{\theta} f(\theta)^{\top} \nabla_{\theta} f(\theta) \nabla_{\mathcal{L}} f(\theta) \quad \& \quad = -\eta \nabla_{\theta} f(\theta)^{\top} \nabla_{\theta} f(\theta) \nabla_{\mathcal{L}} f(\theta) \quad \& \quad = -\eta K(\theta) \nabla_{\mathcal{L}} f(\theta) \quad \& \quad = -\eta K_{\infty} \nabla_{\mathcal{L}} f(\theta) \quad \& \quad \text{for infinite width network}$$

To track the evolution of θ in time, let's consider it as a function of time step t . With Taylor expansion, the network learning dynamics can be simplified as:

$$f(\theta(t)) \approx f^{\text{lin}}(\theta(t)) = f(\theta(0)) + \underbrace{\nabla_{\theta} f(\theta(0))}_{\text{formally } \nabla_{\theta} f(\mathbf{x}; \theta)} \big|_{\theta=\theta(0)} (\theta(t) - \theta(0))$$

Such formation is commonly referred to as the *linearized* model, given $\theta(0)$, $f(\theta(0))$, and $\nabla_{\theta} f(\theta(0))$ are all constants. Assuming that the incremental time step t is extremely small and the parameter is updated by gradient descent:

$$\begin{aligned} \theta(t) - \theta(0) &= -\eta \nabla_{\theta} \mathcal{L}(\theta) \\ &= -\eta \nabla_{\theta} f(\theta)^{\top} \nabla_{\mathcal{L}} f(\theta) \approx f^{\text{lin}}(\theta(t)) - f(\theta(0)) \\ &= -\eta \nabla_{\theta} f(\theta(0))^{\top} \nabla_{\mathcal{L}} f(\mathbf{X}; \theta(0)) \nabla_{\mathcal{L}} f(\theta) \\ &\approx \frac{df(\theta(t))}{dt} = -\eta K(\theta(0)) \nabla_{\mathcal{L}} f(\theta) \\ &\approx -\eta K_{\infty} \nabla_{\mathcal{L}} f(\theta) \quad \& \quad \text{for infinite width network} \end{aligned}$$

Eventually we get the same learning dynamics, which implies that a neural network with infinite width can be considerably simplified as governed by the above linearized model ([Lee & Xiao, et al. 2019](#)).

In a simple case when the empirical loss is an MSE loss, $\nabla_{\theta} \mathcal{L}(\theta) = f(\mathbf{X}; \theta) - \mathcal{Y}$, the dynamics of the network becomes a simple linear ODE and it can be solved in a closed form:

$$\begin{aligned} \frac{df(\theta)}{dt} &= -\eta K_{\infty} (f(\theta) - \mathcal{Y}) \\ \frac{dg(\theta)}{dt} &= -\eta K_{\infty} g(\theta) \quad \text{let } g(\theta) = f(\theta) - \mathcal{Y} \\ \int \frac{dg(\theta)}{g(\theta)} &= -\eta \int K_{\infty} dt \quad \& \quad g(\theta) = C e^{-\eta K_{\infty} t} \quad \& \quad \end{aligned}$$

When $t=0$, we have $C=f(\theta(0)) - \mathcal{Y}$ and therefore,

$$f(\theta) = (f(\theta(0)) - \mathcal{Y})e^{-\eta K_{\infty} t} + \mathcal{Y} \\ = f(\theta(0))e^{-\eta K_{\infty} t} + (I - e^{-\eta K_{\infty} t})\mathcal{Y}$$

5.4 Lazy Training#

People observe that when a neural network is heavily over-parameterized, the model is able to learn with the training loss quickly converging to zero but the network parameters hardly change. *Lazy training* refers to the phenomenon. In other words, when the loss $\mathcal{L}$ has a decent amount of reduction, the change in the differential of the network f (aka the Jacobian matrix) is still very small.

Let $\theta(0)$ be the initial network parameters and $\theta(T)$ be the final network parameters when the loss has been minimized to zero. The delta change in parameter space can be approximated with first-order Taylor expansion:

$$\begin{aligned} \hat{y} &= f(\theta(T)) \approx f(\theta(0)) + \nabla_{\theta} f(\theta(0)) (\theta(T) - \theta(0)) \\ \text{Thus } \Delta \theta &= \theta(T) - \theta(0) \approx \frac{\|\hat{y} - f(\theta(0))\|}{\|\nabla_{\theta} f(\theta(0))\|} \end{aligned}$$

Still following the first-order Taylor expansion, we can track the change in the differential of f :

$$\begin{aligned} \nabla_{\theta} f(\theta(T)) &\approx \nabla_{\theta} f(\theta(0)) + \nabla^2_{\theta} f(\theta(0)) \Delta \theta \\ &= \nabla_{\theta} f(\theta(0)) + \nabla^2_{\theta} f(\theta(0)) \frac{\|\hat{y} - f(\theta(0))\|}{\|\nabla_{\theta} f(\theta(0))\|} \\ \text{Thus } \Delta \big(\nabla_{\theta} f\big) &= \nabla_{\theta} f(\theta(T)) - \nabla_{\theta} f(\theta(0)) = \frac{\|\hat{y} - f(\theta(0))\|}{\|\nabla_{\theta} f(\theta(0))\|} \nabla^2_{\theta} f(\theta(0)) \end{aligned}$$

Let $\kappa(\theta)$ be the relative change of the differential of f to the change in the parameter space:

$$\kappa(\theta) = \frac{\Delta \big(\nabla_{\theta} f\big)}{\|\nabla_{\theta} f(\theta(0))\|} = \frac{\|\hat{y} - f(\theta(0))\|}{\|\nabla_{\theta} f(\theta(0))\|} \frac{\|\nabla^2_{\theta} f(\theta(0))\|}{\|\nabla_{\theta} f(\theta(0))\|^2}$$

Chizat et al. (2019) showed the proof for a two-layer neural network that $\mathbb{E}[\kappa(\theta)] \rightarrow 0$ (getting into the lazy regime) when the number of hidden neurons $\rightarrow \infty$. Also, recommend [this post](#) for more discussion on linearized models and lazy training.

6 Citation#

Cited as:

Weng, Lilian. (Sep 2022). Some math behind neural tangent kernel. Lil'Log.
<https://lilianweng.github.io/posts/2022-09-08-ntk/>.

Or

```
@article{weng2022ntk,
  title = "Some Math behind Neural Tangent Kernel",
  author = "Weng, Lilian",
  journal = "Lil'Log",
  year = "2022",
  month = "Sep",
  url = "https://lilianweng.github.io/posts/2022-09-08-ntk/"
}
```

7 References#

- [1] Jacot et al. “Neural Tangent Kernel: Convergence and Generalization in Neural Networks.” NeurIPS 2018.
- [2] Radford M. Neal. “Priors for Infinite Networks.” Bayesian Learning for Neural Networks. Springer, New York, NY, 1996. 29-53.
- [3] Lee & Bahri et al. “Deep Neural Networks as Gaussian Processes.” ICLR 2018.
- [4] Chizat et al. “On Lazy Training in Differentiable Programming” NeurIPS 2019.
- [5] Lee & Xiao, et al. “Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent.” NeurIPS 2019.
- [6] Arora, et al. “On Exact Computation with an Infinitely Wide Neural Net.” NeurIPS 2019.
- [7] (YouTube video) “Neural Tangent Kernel: Convergence and Generalization in Neural Networks” by Arthur Jacot, Nov 2018.
- [8] (YouTube video) “Lecture 7 - Deep Learning Foundations: Neural Tangent Kernels” by Soheil Feizi, Sep 2020.
- [9] “Understanding the Neural Tangent Kernel.” Rajat’s Blog.
- [10] “Neural Tangent Kernel.” Applied Probability Notes, Mar 2021.
- [11] “Some Intuition on the Neural Tangent Kernel.” inFERENCe, Nov 2020.

- [Foundation](#)
- [Neural-Tangent-Kernel](#)
- [Learning-Dynamics](#)

[Large Transformer Model Inference Optimization](#)
[Generalized Visual Language Models](#)

© 2025 Lil'Log Powered by [Hugo](#) & [PaperMod](#)

8 Citation

Please cite this work as:

Weng, Lilian. “[Title](#)”. Lil 'Log (May 2025). <https://lilianweng.github.io/posts/2022-09-08-ntk/>

Or use the BibTeX citation:

```
@article{weng2025,  
title = {},  
author = {Weng, Lilian},  
journal = {lilianweng.github.io},  
year = {2025}, month = {May},  
url = {}  
}
```

9 References

Tags: [tag1](#) [tag2](#)

[⟨ Title Title ⟩](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)